

Associate Partner Portal — Master Development Instructions

You are developing a completely standalone Associate Partner Portal.

1. Project Independence

This application is completely independent.

DO NOT connect or integrate with:

- CRM
- EMS
- Customer Portal
- Existing CRM databases
- Existing EMS databases
- External business APIs
- Third-party business systems

Do not invent integrations that were not requested.

The portal must have its own application structure, authentication, authorization, database models, workflows and UI.

2. Design Source of Truth

A UI/UX Design Guide PDF will be provided inside the project folder.

Before implementing the interface:

1. Locate the Design Guide PDF.
2. Read and analyze it completely.
3. Extract:
 - Brand colors
 - Typography
 - Font hierarchy
 - Spacing
 - Border radius

- Card style
 - Button style
 - Navigation style
 - Icon style
 - Form style
 - Table style
 - Modal style
 - Dashboard style
 - Responsive behavior
 - Visual hierarchy
4. Treat the PDF as the primary visual reference.
 5. Do not introduce an unrelated design system.
 6. Keep the entire application visually consistent with the PDF and provided logo.

If a requirement conflicts with the PDF aesthetically, preserve the functionality but adapt its visual implementation to the design language of the PDF.

3. Product Roles

There are exactly three primary roles:

1. MD
2. Associate Manager
3. Associate

Do not create unnecessary business roles.

4. Development Method

Work strictly phase-by-phase.

For every phase:

1. Inspect the existing project.
2. Understand the current implementation.
3. Implement only the requirements for the current phase.
4. Run the application.
5. Run all available automated tests.

6. Perform type checking.
 7. Perform linting.
 8. Check console errors.
 9. Check runtime errors.
 10. Check broken routes.
 11. Check responsive behavior.
 12. Test the relevant user workflows manually where possible.
 13. Fix every discovered error.
 14. Run the tests again.
 15. Do not proceed while known errors remain.
 16. Do not break functionality from previous phases.
 17. Only after the current phase passes verification should the next phase begin.
-

5. Testing Rule

After every phase produce an internal verification checklist containing:

- Build status
- TypeScript/type-check status
- Lint status
- Unit test status
- Integration test status
- Runtime status
- Console error status
- Route status
- Responsive status
- Authentication status where applicable
- Role permission status where applicable
- Database status where applicable

If any test fails:

1. Identify the root cause.
2. Fix the root cause.
3. Run the failed test again.
4. Run the complete phase test suite again.
5. Continue only when all tests pass.

Never hide errors with:

- **any**
- disabling lint rules

- suppressing TypeScript errors
 - removing tests
 - commenting out broken functionality
 - swallowing exceptions
 - fake success messages
-

6. Data Integrity

Important records must not be casually deleted.

Prefer statuses such as:

- Active
- Inactive
- Pending
- Approved
- Rejected
- Cancelled
- Archived

Important historical records such as bookings, commissions, reviews, approvals and travel claims must remain traceable.

7. Security

Do not rely only on hiding buttons in the UI.

Authorization must also be enforced at the application/data-access layer.

An Associate must never be able to access another user's restricted data by changing:

- URL parameters
- IDs
- request payloads
- query parameters
- client-side state

Sensitive information includes:

- Aadhaar
 - PAN
 - Bank account
 - IFSC
 - Emergency contact
 - Private address
 - Other confidential personal information
-

8. MD and Associate Manager Security

MD and Associate Manager accounts are fixed accounts.

They use:

- Fixed User ID
- Initial temporary password

On first successful login:

- Force password change.
- Do not allow normal application access until password is changed.

MD and Associate Manager accounts must support single-device active login.

If an account is already active on another device:

- Reject the new login.
- Clearly explain that another active session exists.

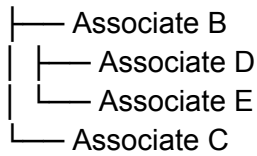
The profile must provide device/session information and an appropriate logout/revoke mechanism.

9. Associate Team Hierarchy

The system must support hierarchical teams.

Example:

Associate A



A can see bookings of A, B, C, D and E.

B can see B, D and E.

B cannot see A or C.

D can see D and its own future downline.

This rule must be enforced in backend/data authorization, not only visually.

10. Mobile / PWA Requirement

The portal must be responsive.

It must work correctly on:

- Desktop
- Laptop
- Tablet
- Mobile

The application should be installable as a web application/PWA.

In My Profile provide:

Install App

When installation is available:

- Show an Install App button.
- Trigger the browser's PWA installation flow where supported.

When the browser/platform does not expose the native install prompt:

- Show appropriate installation instructions.

The installed application must open as a standalone web application rather than simply looking like a normal browser tab.

Do not falsely display "Installed" if the application is not installed.

The PWA must include:

- Web app manifest
- App icons
- Appropriate theme color
- Standalone display mode
- Service worker where appropriate
- Installability requirements
- Responsive mobile UI

Do not implement a fake native mobile application.

11. Mobile Browser Requirement

The same web application must remain fully usable directly in a mobile browser.

Therefore support both:

Browser:

<https://application/...>

Installed PWA:

[Installed Associate Portal](#)

Both must use the same application and data.

Do not create a separate mobile codebase unless technically necessary.

12. UI Quality Standard

The UI must be premium and production-quality.

Avoid:

- Poor alignment
- Excessive empty space
- Inconsistent spacing
- Random colors
- Inconsistent buttons
- Inconsistent border radius
- Generic admin-dashboard appearance
- Unnecessary gradients
- Excessive animations
- Emoji used as primary navigation icons
- Broken mobile layouts

Use:

- Consistent icon system
 - Clear typography hierarchy
 - Proper responsive grids
 - Skeleton loading
 - Empty states
 - Error states
 - Confirmation dialogs
 - Toast notifications
 - Accessible forms
 - Keyboard-friendly interactions
 - Proper hover/focus/active states
 - Smooth but restrained animations
-

13. Important Rule About Scope

Do not add major business functionality that has not been requested.

You may add small UX improvements that are clearly necessary for usability, security or accessibility.

If a major business decision is missing, stop and identify it rather than inventing a complex workflow.

14. Completion Rule

At the end of every phase:

Do not simply say "completed."

Provide:

PHASE STATUS

- Implemented
- Tested
- Errors found
- Errors fixed
- Final test result
- Files/modules changed
- Remaining known issues, if any

Only mark the phase complete if there are no known blocking errors.

After completing a phase, wait for the next phase instruction.